
Modernize development workflow with VS code with SSH extension

**Remote Development: From
WinSCP to VS Code with SSH**

by Jana :)

Current Workflow with WinSCP

- GUI-based file transfer between local & remote (A file explorer-like view of your **local files** on one side; The **remote server files** on the other side)
- Limited integration with editors or terminals
- Manual file syncing, lacks real-time collaboration
- Not ideal for development at scale

● *“WinSCP is great for quick file transfers, but not for developing efficiently on remote systems.”*

Example of development Workflow Today (with WinSCP):

1. Connect to remote server using WinSCP
2. Navigate to a file, download it
3. Open it in local editor (PyCharm / VS Code)
4. Make changes
5. Upload the file back
6. Switch to PuTTY/SSH client to test or run it
🔄 Repeat

🟡 *“A lot of context switching and potential for error.”*

Why VS Code + SSH?

- Seamless access to remote files **as if they're local**
- Built-in terminal access to remote shell
- Real-time development using extensions, linting, Git
- Secure, fast, and more intuitive for developers

● *“Edit, build, and debug directly on the remote server—no manual file syncing.”*

Demo Flow / How It Works

1. Install the **Remote - SSH** extension in VS Code
2. Open the **Command Palette** (**Ctrl+Shift+P**) → *Remote-SSH: Connect to Host*
3. Enter or select the remote host (e.g., `user@remote-ip` ili `ec2-trift-machine`)
4. VS Code opens a remote workspace
5. Develop, test, run – all remotely!

💡 Tip: You can save host configs in `~/.ssh/config` for quick access. (**Ctrl+Shift+P**) → *Remote-SSH: Open Configuration File* and select `.ssh/config`

Remote SSH via VS Code – First-Time Setup

- Open VS Code
 - Go to **Extensions** (**Ctrl+Shift+X**)
 - Search for “**Remote - SSH**” by Microsoft and install it
 - **Make Sure You Have SSH Access to the Remote Machine**
 - You need:
 - **username**
 - **remote IP or hostname**
 - Either a **password** or an **SSH key** (**id_rsa**)
 - A **PuTTY PPK (PuTTY Private Key)** cannot be directly used in the **~/.ssh/config** file, as OpenSSH requires keys in **PEM format**. You need to convert your **PPK key** to an **OpenSSH-compatible key** before using it. (on the next slide)
 - Update **~/.ssh/config**
 - Connect to the Server
 - **ssh myserver**
-

Convert PPK to OpenSSH Format on Windows - first time setup

You need to use **puttygen** to convert your PPK key.

1. Open **PuTTYgen**.
2. Click **Load** and select your **.ppk** file.
3. Click **Conversions** → **Export OpenSSH key**.
4. Save it as **id_rsa** (or any filename) in **~/ .ssh/** on your local machine.
 - i. Make sure **"Save as type"** is set to **All Files**.
 - ii. If PuTTYgen asks, do **not** add a passphrase unless required.
5. Open Command Prompt and Ensure the permissions are correct:

```
icacls C:\Users\YourUsername\.ssh\id_rsa /inheritance:r
```

```
icacls C:\Users\YourUsername\.ssh\id_rsa /grant:r "%USERNAME%:F"
```

Now - Live Demo

Show quick setup: connecting to a server

Open a file, make a change, run a script

Highlight seamless experience

.ssh/config moj fajl

```
Host ec2-trift-machine
    HostName 10.224.37.252
    User ubuntu
    IdentityFile ~/.ssh/id_rsa
    ForwardAgent yes
    StrictHostKeyChecking no
    UserKnownHostsFile=/dev/null
```
